Structured programming → Functions

OOP → creating objects (represents real world things)

Object → Data

→ Functions

Class: Blueprint that describes common features of a group of objects | Describes what object will look like

An object is created from a class.

e.g: The Blueprint says car have 4 wheels, a speed limit, a color ....

From this Blueprint we make real objects e.g

A red Toyota

Syntax

```
class   classname
{
        Class member list →  members of class
};
```

```
class Student {
    int age;
    String name;
};
```

We can declare variables inside a class But not assign values to them in the class. (Can define outside

you can put a function prototype in the class also.

```
class Student {
    int age;
    String name;
    void showInfo();   → member function
};
```

If a function definition is written inside the class then we can use the class var inside the functn without needing to pass the arguments.

Members type

→ Private (default)
    can only be used inside the class

By default members are private.

→ public: can be used anywhere

→ Protected → can only be used by class itself and child classes

---

example:

```
class person {
        char name[20];
        int id;
      Public:
        void getdetails() {
            cout<<"hello";
        }
```

```
};

int main() {
    person p;        → object
    p.getdetails();
    p.x;
}
```

Declaring a object

classname objectname;

Accessing specific members of class objectname.member;

```
class myclass {
public:
    int myNum;
    string mystring;
    void fun();      member function
};
```

Return type classname :: funname() {
}

```
void myclass::fun()
{
    cout<<"hello";
}
```

declare outside the class

completely declared outside the class

Scope resolution operator is used to connect function to the class. / hence only then we can use the class members inside the function.

classes / functions are declared above int main

(Built in operations) like +, -, ==, if() we can perform these operations only on class members.

(Accessor Function and mutator functions)

```
class Student {
    public:
        int marks;
};
```

Now in main

```
Student S
S.marks = -100;
```

(X) this makes no sense.

Because marks cannot be neg

For this we make the variable private:

```
class Student {
    public:
        int marks;
};
```

Now if you wat to read/modify the privates members

we use mutators (setters) and accessors (getters)

- mutators (setters) → only Sets/change values of private data
- accessors (getters) → only Read values of private data/shows → return

Utility function → helper function inside of class. | privite function of the class.

Constructor → Special function inside of a class. Used to initialise objects.
→ Same name as the class
→ No Return type (not even void)

Default constructor → is a constructor with no parameters.
(it is a must to implement this first)

```
class car {
    public :
        car() {
            cout<< "Hello world "<<endl;
        }
};

int main() {
    car c1;
}
```

Output will be Hello world

Constructor → Automatically sets up the object by intialising the class members.

Syntax

```
classname(){
    //////
    //////
}
```

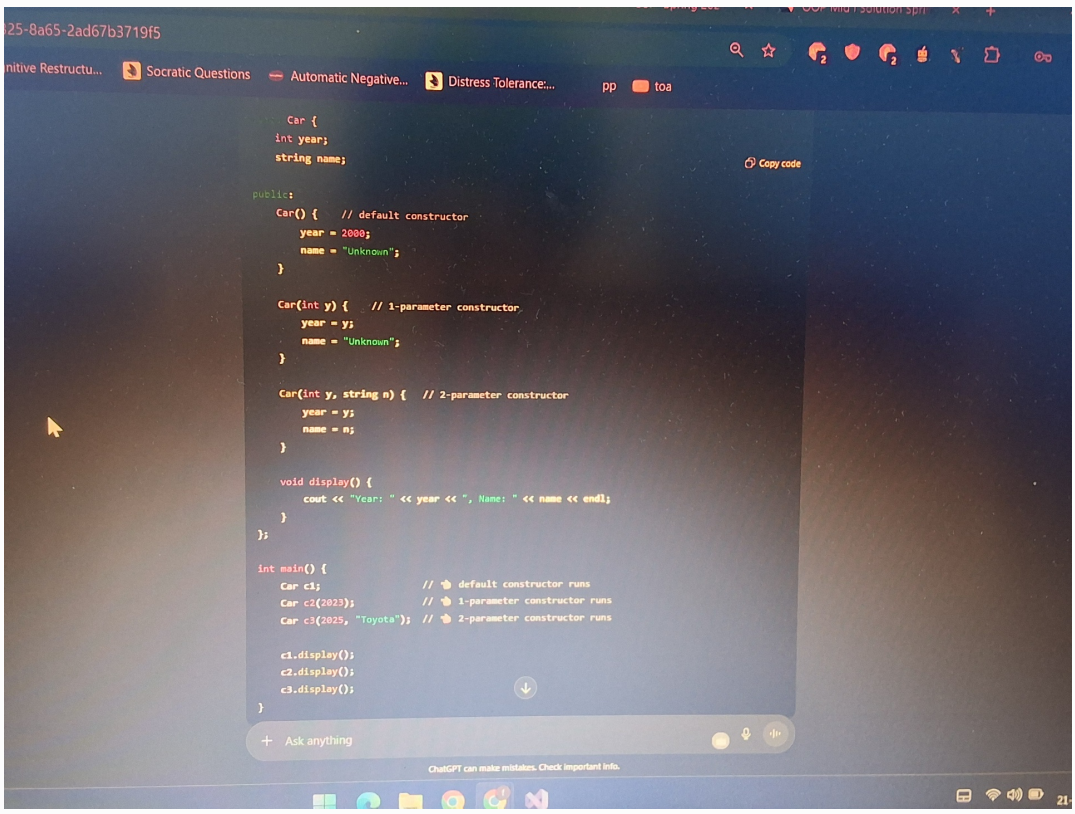
constructor is always public

So whatever is written inside the constructor is automatically executed when the object is created.

A class can hold more than 1 constructor !!! and what differentiates constructor from another constructor is parameters list.

```
Car ();
Car (int y);
car (int y, int p);
```

For each constructor we make its own new object depending on the parameters




```
Car {
int year;
string name;

public:
Car() {    // default constructor
    year = 2000;
    name = "Unknown";
}

Car(int y) {    // 1-parameter constructor
    year = y;
    name = "Unknown";
}

Car(int y, string n) {   // 2-parameter constructor
    year = y;
    name = n;
}

void display() {
    cout << "Year: " << year << ", Name: " << name << endl;
}
};

int main() {
    Car c1;              // default constructor runs
    Car c2(2023);        // 1-parameter constructor runs
    Car c3(2025, "Toyota"); // 2-parameter constructor runs

    c1.display();
    c2.display();
    c3.display();
}
```

→ constructor overloading

copy constructor: A special constructor that creates a new object by copying data from an existing object.

↓

takes an existing object and copies its data into a new object

Syntax: classname (classname &ref)

↓ ref variable to the object we want to copy data from and into our new object

```
class A {
    int a,b;
    Public:
    A (int x, int y)
    {
        a=x;  b=y;
    }
    A (A &ref)
    {
        a = ref.a;   b = ref.b;
    }
};
int main () {
    A obj (10, 20), obj3
    A obj1 = obj;
    or
    A obj2(obj);
}
```

e.g. Book book3 (book1) → everything that is in book 1 copy that into book 3

b = ref.b; → gets passed in obj1 → value of obj

A obj1 = obj; → copy constructor

A obj2(obj); Same thing

obj3 = obj (X) error the obj3 must be created at the same line.

↓ copy constructo will not be called.

Shallow copy ⇒ only address copied.

Deepcopy → new addr value is copi

Default parameters

Wall (double Len, double height = 2);

(give default values from Rightmost parameters)

Destructor:: → (is a special function in a class)

↳ cleanup function automatically called when an object ends.

(Name begins with ~ and must match the class name)

```
class Geeks {

    ~Geeks()

};
```

we cannot overload a destructor, only one allowed per class.

→ no parameters are allowed

→ no return type of destructors.

Destructor → last chance to free memory before the object is gone

↓

Public

Distructor runs automatically when an object goes out of scope

So whenever the program reaches } object is automatically destroyed

```
class num {
    public:
    num() {
        cout << "hello";
    }
    ~num() {
        cout << "Destructor called";
    }
};

int main() {
    cout << "Entering in main";
    num N1;
    {
        cout << "Entering a new block";
        num N2, N3;
        cout << "Exiting";
    }
    cout << "Back to main";
return 0;
}
```

output:

Entering in main
N1 ← hello
Entering a new block
hello → N2
hello → N3
Exiting
N2 ← Destructor called
N3 ← Destructor called
Back to main
Destructor called
N1

this operator → Stores the address of the object that called the function.

```
class Student {
    int age
    string name
Student (string name , int age) {
    this -> name = name;
    this -> age = age;
}
}
```

Note:: A object can be created inside a method (Same class) or as a data member of Another class. But cannot be created as a data member of the Same class. (Recursion)

Non-Static members :: Belong to each object seperately.

```
class student {
public:
    string name;
};
```

Static members :: all objects use the Same memory that is Static

↓ Belongs to class and not the objects

```
class student {
public:
    static int count;
};
int student :: count = 1 ⇒ Intializing the static member
```

By default it is 0.

Static data item is useful when all objects of the Same class share a common item.

outside the class we have to define Intialize all static members.

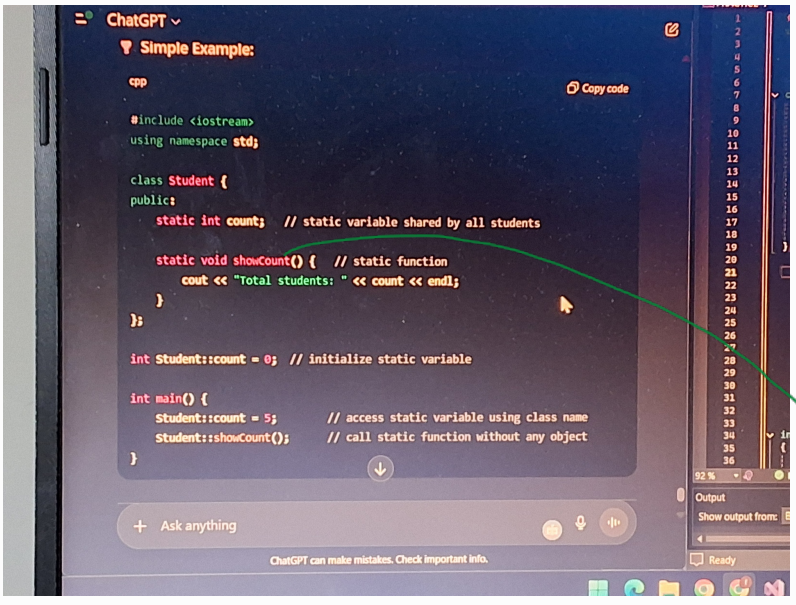
returntype classname :: varname ;

Static fuction → Belongs to a class and not the individual object, can be called without creating any objects and is used to access static variables

To call it in main

Syntax Class :: function name ( ) ;

note: A static function has access to only static members and not other normal var.



Static function

Static function → shared function for all objects not just one.

Note:
every time a constructor runs an object has been created.

and utility functions are static VVIMP

Move constructor: allows an object to move (resources) from another object without creating a copy of that object.

↓

faster

Syntax:

```
classname (classname && other) {
        take resources
}
```

```
class Bucket {
    Public:
        int water;
        Bucket(int w): water(w){};
        Bucket (Bucket &f other) {
            water = other.water;
            other.water = 0
        }
};
```

In main we can do

```
Bucket a(10);
Bucket b(move(a));
```

this is called.

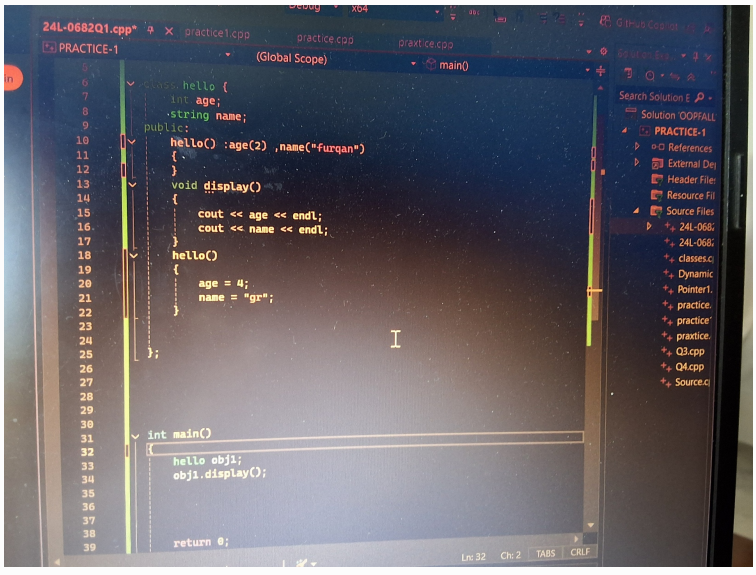
if we write

```
Bucket b(a)
```

↓

this calls copy constructor.

Setting up a constructor using Intializer list



Syntax for Intializalia list in constructo

constructor (para) : Intialisation Section

{

assignment code;

}

class Test {

int a;

int b;

Public:

Test (int i, int j) : a(i) , b(j)

{

cout << " constructor Body";

}

i values goes to a

j value goes to b

Test (int i, int j): b(j), a(i) will not work as we declared a first so that must be Intialized first

{

cout << " constructor Body";

}

(Cascaded Functions) → each method returns a ref

```
int main() {
    Rec T1;
    T1.dis1();
    T1.dis2();
    T1.dis3();
}
```

long:

T1.dis1().dis2().dis3() ✓

```
class rec() {
    Private
        int width;
        String color;
    Public:
        rec & Setvalues(int len, String value)
            {
                width = len;
                color = value;
                return *this → returns the object itself
            }
```

now ✓

returning the object itself and not a copy so that when lets say dis2() is envoked for e.g the method is applies to the same object and not a copy so it doesn't break the chaining

T1.dis1().dis2().dis3()